

Python Code

```
1  #Fibonacci (DP - Bottom-Up)
2  def fibonacci(n):
3      dp = [0]*(n+1)
4      dp[0] = 0
5      dp[1] = 1
6
7      for i in range(2, n+1):
8          dp[i] = dp[i-1] + dp[i-2]
9
10     return dp[n]
11
12     print(fibonacci(10))
13     #Fibonacci (Top-Down Memoization)
14     def fib(n, memo={}):
15         if n <= 1:
16             return n
17
18         if n in memo:
19             return memo[n]
20
21         memo[n] = fib(n-1, memo) + fib(n-2, memo)
22         return memo[n]
23
24     print(fib(10))
25     #3. Climbing Stairs (similar to Fibonacci)
26     def climb_stairs(n):
27         dp = [0]*(n+1)
28         dp[0] = 1
29         dp[1] = 1
30
31         for i in range(2, n+1):
32             dp[i] = dp[i-1] + dp[i-2]
33
34         return dp[n]
35
36     print(climb_stairs(5))
37     #4. 0/1 Knapsack (Classic Beginner DP)
38     ##Weights + values → maximize profit.
39     def knapsack(W, wt, val, n):
40         dp = [[0]*(W+1) for _ in range(n+1)]
41
42         for i in range(n+1):
43             for w in range(W+1):
44                 if i == 0 or w == 0:
45                     dp[i][w] = 0
46                 elif wt[i-1] <= w:
47                     dp[i][w] = max(
48                         val[i-1] + dp[i-1][w-wt[i-1]],
49                         dp[i-1][w]
50                     )
51                 else:
52                     dp[i][w] = dp[i-1][w]
```

```
53
54     return dp[n][W]
55
56 print(knapsack(7, [1,3,4,5], [1,4,5,7], 4))
57 #5. Subset Sum (DP Table)
58 def subset_sum(arr, target):
59     n = len(arr)
60     dp = [[False]*(target+1) for _ in range(n+1)]
61
62     for i in range(n+1):
63         dp[i][0] = True
64
65     for i in range(1, n+1):
66         for s in range(1, target+1):
67             if arr[i-1] <= s:
68                 dp[i][s] = dp[i-1][s] or dp[i-1][s-arr[i-1]]
69             else:
70                 dp[i][s] = dp[i-1][s]
71
72     return dp[n][target]
73
74 print(subset_sum([2,3,7,8,10], 11))
75 #6. Coin Change (minimum coins)
76 def coin_change(coins, amount):
77     dp = [float("inf")]*(amount+1)
78     dp[0] = 0
79
80     for coin in coins:
81         for i in range(coin, amount+1):
82             dp[i] = min(dp[i], dp[i-coin] + 1)
83
84     return dp[amount] if dp[amount] != float("inf") else -1
85
86 print(coin_change([1,2,5], 11))
87 #7. Longest Increasing Subsequence (LIS) – O(n²)
88 def lis(arr):
89     n = len(arr)
90     dp = [1]*n
91
92     for i in range(n):
93         for j in range(i):
94             if arr[j] < arr[i]:
95                 dp[i] = max(dp[i], dp[j] + 1)
96
97     return max(dp)
98
99 print(lis([10, 22, 9, 33, 21, 50]))
100 # 8. Longest Common Subsequence (LCS)
101 def lcs(x, y):
102     n = len(x)
103     m = len(y)
104     dp = [[0]*(m+1) for _ in range(n+1)]
105
106     for i in range(1, n+1):
107         for j in range(1, m+1):
```

```

108         if x[i-1] == y[j-1]:
109             dp[i][j] = dp[i-1][j-1] + 1
110         else:
111             dp[i][j] = max(dp[i-1][j], dp[i][j-1])
112
113     return dp[n][m]
114
115 print(lcs("abcde", "ace"))
116 # ★ 9. Edit Distance (Levenshtein)
117 def edit_distance(a, b):
118     n, m = len(a), len(b)
119     dp = [[0]*(m+1) for _ in range(n+1)]
120
121     for i in range(n+1):
122         dp[i][0] = i
123     for j in range(m+1):
124         dp[0][j] = j
125
126     for i in range(1, n+1):
127         for j in range(1, m+1):
128             if a[i-1] == b[j-1]:
129                 dp[i][j] = dp[i-1][j-1]
130             else:
131                 dp[i][j] = 1 + min(
132                     dp[i-1][j],
133                     dp[i][j-1],
134                     dp[i-1][j-1]
135                 )
136
137     return dp[n][m]
138
139 print(edit_distance("horse", "ros"))
140 # ★ 10. House Robber (cannot take adjacent)
141 def rob(nums):
142     if len(nums) == 1:
143         return nums[0]
144
145     dp = [0]*len(nums)
146     dp[0] = nums[0]
147     dp[1] = max(nums[0], nums[1])
148
149     for i in range(2, len(nums)):
150         dp[i] = max(nums[i] + dp[i-2], dp[i-1])
151
152     return dp[-1]
153
154 print(rob([1,2,3,1]))
155
156 #####33
157 dic = {}
158 def subsets(lst, sums):
159
160     if sums == 0:
161         return True
162     if len(lst) == 0 or sums < 0:

```

```

163         return False
164     key = (tuple(lst), sums)
165     if key in dic:
166         return dic[key]
167
168     include = subsets(lst[1:], sums - lst[0])
169     exclude = subsets(lst[1:], sums)
170     result = (include or exclude)
171     dic[key] = result
172
173     return result
174
175     lst = [5, 3, 11, 8, 2]
176     sums = 16
177     result = subsets(lst, sums)
178     print(result)
179     lst = [2, 3, 34, 11, 1]
180     sums = 6
181     dp = []
182     ns = len(lst)
183     for i in range(ns+1):
184         row = []
185         for j in range(sums+1):
186             if j == 0:
187                 row.append(True)
188             else:
189                 row.append(False)
190         dp.append(row)
191     for i in dp:
192         print(i)
193     print("_____")
194     n = len(dp)
195     for i in range(1, n):
196         for j in range(1, sums+1):
197             results = False
198             if dp[i-1][j] == True:
199                 results = True
200             else:
201                 if j - lst[i-1] >= 0 and dp[i-1][j - lst[i-1]] == True:
202                     results = True
203
204
205             dp[i][j] = results
206
207     for i in dp:
208         print(i)
209
210

```